

Sentrix Whitepaper (Bahasa Indonesia)

Sentrix

Blockchain Layer-One untuk Ekonomi Riil

Abstrak

1. Pendahuluan
2. Visi, Misi, dan Alasan Keberadaan
 - 2.1 Visi
 - 2.2 Misi
 - 2.3 Mengapa Sentrix Ada
 - 2.4 Mengapa Sekarang
 - 2.5 Mengapa Indonesia Pertama
3. Prinsip Desain
4. Arsitektur
 - 4.1 Lapisan Konsensus
 - 4.2 Lapisan Eksekusi
 - 4.3 Lapisan State
 - 4.4 Model Jaringan
 - 4.5 Format Transaksi
5. Operasi Native
 - 5.1 Pertanyaan Native vs Kontrak
 - 5.2 Set Operasi Native
 - 5.3 Siklus Hidup Transaksi
6. Tokenomik
 - 6.1 Pasokan
 - 6.2 Block Reward dan Halving
 - 6.3 Mekanisme Fee
 - 6.4 Premine dan Distribusi Awal
 - 6.5 Routing Reward
7. Ekonomi Validator
 - 7.1 Stake yang Dipertaruhkan dan Slashing
 - 7.2 Block Reward dan Bagi Hasil Fee
 - 7.3 Penalti Liveness
 - 7.4 Genesis dan Bootstrap
8. Model Keamanan
 - 8.1 Jaminan Kriptografis
 - 8.2 Jaminan Kripto-Ekonomi
 - 8.3 Jaminan Sosial
 - 8.4 Keamanan Kuantitatif
 - 8.5 Long-Range Attacks dan Weak Subjectivity
 - 8.6 Mode Kegagalan
9. Tesis Ekonomi Riil
10. Perbandingan dengan Karya Sebelumnya
11. Arah ke Depan
12. Kesimpulan
- Lampiran A — Parameter Protokol
- Referensi
- Penghargaan

Sentrix

Blockchain Layer-One untuk Ekonomi Riil

Penulis: Satya Kwok **Versi:** 1.1

Abstrak

Kami memperkenalkan Sentrix, sebuah blockchain Layer-One yang dioptimalkan untuk penyelesaian aktivitas ekonomi dunia riil. Sentrix menggunakan seleksi validator delegated-proof-of-stake yang dipadukan dengan protokol persetujuan Byzantine Fault Tolerant tiga fase untuk menghasilkan blok dengan finalitas satu detik. Operasi

protokol native—penerbitan token, staking, koordinasi validator—dieksekusi langsung terhadap state kanonis, sementara jalur eksekusi kedua menjalankan Ethereum Virtual Machine untuk programabilitas umum. Kebijakan moneter ditetapkan: pasokan maksimum 315 juta SRX, block reward satu SRX yang halving setiap kira-kira 126 juta blok (empat tahun pada blok satu detik), dan mekanisme fee yang menghancurkan 50% setiap fee transaksi selamanya. Chain ini dirancang sebagai infrastruktur keuangan tahan lama untuk aktivitas ekonomi dunia riil, dimulai di Indonesia dan berkembang ke luar. Makalah ini menentukan rasional desain, arsitektur, siklus hidup transaksi, properti keamanan konsensus, mekanisme moneter, dan model ancaman.

1. Pendahuluan

Sebagian besar blockchain publik dirancang untuk hal lain selain melayani aktivitas ekonomi riil. Bitcoin dirancang sebagai sistem kas elektronik peer-to-peer [1], namun pada praktiknya beroperasi terutama sebagai lapisan penyelesaian untuk penyimpanan nilai. Ethereum dirancang sebagai komputer dunia [2], namun ekonomi on-chain-nya didominasi oleh spekulasi finansial, perdagangan on-chain, dan aset sintetis. Solusi penskalaan Layer-Two mewarisi orientasi ini dan memperbesarnya.

Disconnect antara infrastruktur blockchain dan aktivitas ekonomi riil mayoritas populasi dunia sangat mencolok. Sebagian besar perdagangan manusia masih didominasi dalam mata uang lokal, diselesaikan melalui jalur perbankan yang tidak mengalami perbaikan substansial selama puluhan tahun, dan secara efektif terkucil dari partisipasi blockchain publik. Friksi ini bukan filosofis—bisnis riil, koperasi riil, pedagang lintas-negara riil menginginkan penyelesaian yang cepat dan final—melainkan arsitektural. Infrastruktur yang mereka butuhkan bukan infrastruktur yang sudah dibangun.

Sentrix adalah respons terhadap kesenjangan arsitektural ini. Sentrix adalah blockchain yang setiap pilihan desainnya memprioritaskan penyelesaian ekonomi riil di atas spekulasi perdagangan. Di mana chain serbaguna mengutamakan fleksibilitas maksimum dengan biaya efisiensi, Sentrix menaikkan primitif ekonomi yang paling umum menjadi operasi protokol native, menghilangkan seluruh kelas overhead. Di mana sebagian besar chain mentolerir waktu blok sepuluh atau tiga puluh detik, Sentrix menargetkan finalitas satu detik—cukup untuk interaksi tingkat retail. Di mana banyak chain mengikuti model token inflasioner, Sentrix dibatasi keras, halving, dan deflasioner—setiap fee dibakar sebagiannya selamanya, sehingga pasokan beredar menyusut seiring pertumbuhan aktivitas.

Sentrix dibangun pertama-tama untuk Indonesia, di mana populasi, proporsi unbanked, dan volume remitansi membentuk demand yang besar dan belum terpenuhi.

2. Visi, Misi, dan Alasan Keberadaan

2.1 Visi

Masa depan di mana aktivitas ekonomi dunia riil—pembayaran lintas-negara, lending berbasis aset, perdagangan retail, tabungan koperasi, penyelesaian rantai pasok, verifikasi identitas—berjalan di atas rel yang transparan, berbiaya rendah, dapat diprogram, tidak dimiliki oleh korporasi tunggal, dapat diakses dari koneksi internet mana pun, dan secepat gesekan kartu kredit.

Kami tidak percaya visi ini dapat dicapai melalui retrofit sistem perbankan lama. Kami tidak percaya itu dapat dicapai pada blockchain yang dirancang untuk spekulasi perdagangan. Itu dapat dicapai melalui infrastruktur khusus yang memperlakukan penyelesaian ekonomi riil sebagai concern kelas pertama, bukan afterthought.

Sentrix adalah satu implementasi dari infrastruktur tersebut.

2.2 Misi

Membangun lapisan penyelesaian yang paling andal, biaya rendah, dan mudah diakses untuk aktivitas ekonomi dunia riil, dimulai dari geografi dan kasus penggunaan yang gagal dilayani oleh infrastruktur lama.

Secara konkret, ini berarti:

- Penyelesaian final satu detik untuk transfer, swap, atau pemanggilan kontrak apa pun.
- Biaya transaksi diukur dalam pecahan sen, terlepas dari ukuran transfer.
- Dukungan protokol native untuk penerbitan token, staking, dan manajemen aset tanpa memerlukan deploy smart contract.
- Kompatibilitas dengan toolchain developer Ethereum global sehingga aplikasi yang sudah ada dapat diport tanpa modifikasi.
- Codebase terbuka, operasi transparan, kebijakan moneter deterministik.
- Orientasi pintu masuk pasar terhadap Indonesia dan Asia Tenggara, di mana demand yang belum terpenuhi paling besar.

2.3 Mengapa Sentrix Ada

Masalah infrastruktur yang kami atasi memiliki empat dimensi:

Latensi. Rel pembayaran yang ada menyelesaikan dalam hitungan hari (correspondent banking), jam (jaringan kartu), atau puluhan detik (blockchain yang dioptimalkan untuk keamanan atau perdagangan). Perdagangan riil menuntut konfirmasi sub-detik untuk point-of-sale, retail, dan operasi bisnis rutin.

Biaya. Fee remitansi lintas-negara 5–10% masih umum. Interchange fee jaringan kartu 1–3% mengekstrak nilai pada setiap transaksi retail. Biaya gas smart-contract pada chain serbaguna rutin melebihi nilai yang ditransfer untuk pembayaran kecil. Operasi native Sentrix menargetkan biaya yang diukur dalam pecahan sen US terlepas dari nilai aset yang mendasarinya.

Programabilitas tanpa overhead. Blockchain telah mendemonstrasikan bahwa transaksi yang dapat diprogram memungkinkan konstruksi ekonomi baru yang kuat—lending, escrow, automated market making, verifikasi identitas. Tetapi operasi rutin seperti mentransfer token, mengklaim reward staking, atau mendaftarkan validator seharusnya tidak memerlukan deploy kode smart-contract yang diaudit. Mereka harus menjadi bagian dari protokol.

Inklusi. Infrastruktur perbankan berkorelasi dengan kekayaan nasional. Infrastruktur blockchain publik secara prinsip dapat memutus korelasi tersebut, tetapi pada praktiknya telah mereproduksinya: sebagian besar aktivitas on-chain berasal dari sekumpulan kecil yurisdiksi berpendapatan tinggi. Sentrix diposisikan untuk membalik default ini dengan membangun pertama-tama untuk geografi yang paling tidak dilayani oleh sistem lama.

2.4 Mengapa Sekarang

Tiga prasyarat telah konvergen yang membuat pekerjaan ini feasible.

Pertama, **EVM telah matang menjadi standar.** Tooling, dompet, library smart-contract, konvensi audit keamanan, dan model mental developer telah konvergen. Chain baru yang mengadopsi kompatibilitas EVM mewarisi seluruh ekosistem dengan biaya nol.

Kedua, **konsensus Byzantine Fault Tolerant telah diproduksi.** BFT ala Tendermint telah menjalankan mainnet yang andal selama bertahun-tahun. Bagian sulit dari konsensus sekarang sudah well-trodden, dan chain baru dapat membangun di atas implementasi open-source yang robust.

Ketiga, **Rust telah matang menjadi bahasa sistem yang viable untuk implementasi blockchain.** Jaminan memory safety, runtime async yang matang, library kriptografis yang berperforma, dan

budaya efisiensi zero-copy membuat pengembangan chain solo atau tim kecil feasible dengan cara yang tidak feasible satu dekade lalu.

Chain yang menggabungkan ketiganya—kompatibilitas EVM, konsensus BFT, dan implementasi Rust—dan menerapkannya ke kasus penggunaan ekonomi riil secara teknis feasible hari ini dengan cara yang tidak feasible lima tahun lalu.

2.5 Mengapa Indonesia Pertama

Indonesia adalah negara terpadat keempat di dunia dan ekonomi terbesar di Asia Tenggara. Indonesia memiliki:

- Populasi muda dan mobile-first dengan penetrasi smartphone tinggi.
- Populasi unbanked atau underbanked yang besar (~40% menurut beberapa estimasi).
- Struktur ekonomi informal yang kuat (perbankan koperasi, lending komunitas, microfinance) yang memetakan secara alami ke penyelesaian blockchain.
- Aliran remitansi yang berkembang pada skala puluhan miliar dolar per tahun.
- Lingkungan regulasi yang, walaupun masih matang, telah menunjukkan kesediaan untuk berhubungan dengan crypto dan blockchain.
- Komunitas developer crypto-aware lokal yang berarti.

Kombinasi demand yang besar dan belum terpenuhi, kesiapan teknologi, dan kecocokan budaya menjadikan Indonesia titik awal yang alami. Setelah lapisan penyelesaian tahan lama dan berguna di pasar ini, ekspansi ke pasar yang sebanding di Asia Tenggara dan seterusnya menjadi straightforward.

Ini bukan framing “emerging markets”. Ini framing pasar spesifik. Sentrrix dibangun untuk melayani sekumpulan pengguna tertentu dengan kebutuhan tertentu, dan dirancang untuk berkembang ke luar dari sana.

3. Prinsip Desain

Enam prinsip membentuk setiap keputusan arsitektural di Sentrrix:

1. Penyelesaian di atas spekulasi. Blockchain yang dioptimalkan untuk perdagangan menciptakan struktur insentif yang menghasilkan perdagangan. Blockchain yang dioptimalkan untuk penyelesaian menghasilkan perdagangan ekonomi riil. Kami memilih yang kedua di setiap percabangan.

2. Primitif native di atas overhead kontrak. Operasi yang akan dilakukan oleh hampir setiap pengguna—memegang token, staking, mengklaim reward, memindahkan aset—seharusnya tidak memerlukan deploy atau interaksi dengan smart contract. Mereka adalah bagian dari protokol.

3. Kompatibilitas EVM untuk kasus umum. Di mana programabilitas dibutuhkan, EVM melayaninya dengan baik—standar yang berkembang dengan dukungan tooling yang luas. Sentrrix menjalankan EVM dengan setia, berdampingan dengan operasi native-nya.

4. Kebijakan moneter deflasioner. Token inflasioner memberi imbalan kepada holder awal dengan biaya holder berikutnya. Sentrrix dibatasi pasokan, halving sesuai jadwal yang sama dengan Bitcoin, dan membakar separuh dari setiap fee. Seiring pertumbuhan aktivitas, pasokan menyusut.

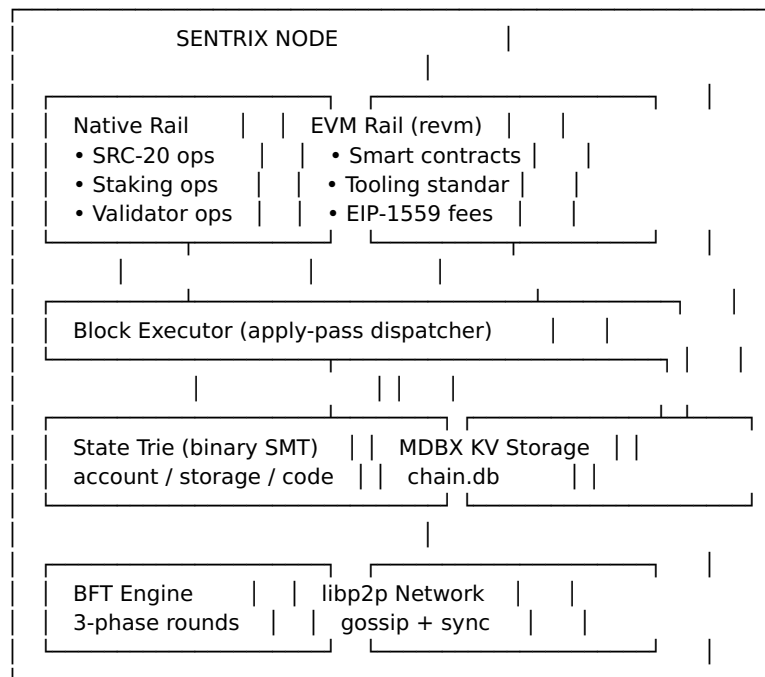
5. Keamanan kripto-ekonomi melalui staking. Desentralisasi adalah properti dari siapa yang dapat melakukan staking, bukan siapa yang saat ini melakukannya. Validator set Sentrrix terbuka, tanpa izin, dan diamankan secara ekonomi oleh stake yang dipertaruhkan.

6. Riil di atas nominal. Aset dunia riil, bisnis riil, aktivitas ekonomi riil. Sentrix menghindari abstraksi on-chain yang nilainya hanya berasal dari abstraksi on-chain lainnya. Chain melayani dunia; dunia tidak melayani chain.

4. Arsitektur

Sentrix terstruktur sebagai empat subsistem terintegrasi yang beroperasi pada satu state kanonis tunggal.

Gambar 1 — Arsitektur sistem



4.1 Lapisan Konsensus

Sentrix menggunakan model seleksi delegated-proof-of-stake yang dipadukan dengan protokol persetujuan Byzantine Fault Tolerant tiga fase [3].

4.1.1 Seleksi Validator

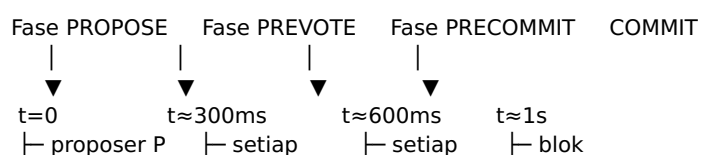
Validator set dipilih oleh delegasi yang dibobotkan stake. Pemegang token mana pun dapat mendelegasikan kepada kandidat validator mana pun; ranking berbobot yang dihasilkan menentukan active set untuk setiap epoch. Keanggotaan active-set dikomputasi ulang pada batas epoch yang tetap, memungkinkan validator baru masuk dan validator yang berkinerja kurang keluar tanpa intervensi governance.

Self-stake minimum untuk validator dilaksanakan pada level protokol. Stake yang didelegasikan diikat dengan periode unbonding: penarikan stake memulai timeout berdurasi tetap di mana stake tetap dapat di-slash tetapi tidak memenuhi syarat untuk reward baru. Ini mencegah validator mengoffload risiko segera sebelum berperilaku buruk.

4.1.2 Ronde Tiga Fase

Dalam epoch, active set menjalankan ronde tiga fase ala Tendermint untuk memfinalisasi setiap blok. Sebuah ronde terdiri dari tiga jenis pesan dan tiga transisi fase yang sesuai:

Gambar 2 — Timing ronde BFT



	untuk height H	validator	validator	difinalisasi
	mengusulkan B	prevote B	precommit B	
		jika valid	jika 2/3+	
			lihat	
			prevote B	

Sebuah blok bersifat final ketika supermayoritas ($\geq \frac{2}{3}$ dari active set yang dibobotkan stake) precommit blok yang sama dalam ronde yang sama. Justifikasi—kumpulan tanda tangan precommit—disertakan di blok berikutnya, memberikan bukti publik bahwa parent telah difinalisasi.

4.1.3 Aturan Locking dan Skip Ronde

Setelah validator prevote sebuah blok dalam ronde, ia mengunci pada blok tersebut. Dalam ronde-ronde berikutnya pada height yang sama, ia hanya akan prevote blok yang berbeda jika menerima “polka” ($\geq \frac{2}{3}$ prevote untuk blok baru) pada ronde yang lebih tinggi. Properti locking inilah yang menjamin keamanan: dua blok yang konflik tidak dapat keduanya mengumpulkan precommit supermayoritas.

Jika ronde gagal difinalisasi dalam timeout (proposer offline, partisi jaringan, ketidakhadiran validator), protokol berlanjut ke ronde berikutnya. Proposer berotasi round-robin melalui active set. Setelah jumlah ronde gagal berurutan yang dapat dikonfigurasi, timeout ronde berlipat ganda, memberikan pemulihan weak-synchrony.

4.1.4 Keamanan dan Liveness

Di bawah asumsi BFT standar bahwa kurang dari sepertiga validator aktif berbobot stake bersifat Byzantine, dua teorema keamanan berlaku:

- **Agreement:** Tidak ada dua validator jujur yang memfinalisasi blok berbeda pada height yang sama.
- **Validity:** Sebuah blok yang difinalisasi adalah well-formed dan apply secara bersih.

Liveness berlaku di bawah asumsi weak-synchrony: pesan antara validator jujur akhirnya tersampaikan, mungkin dengan delay terbatas. Di bawah asumsi ini, protokol menjamin bahwa progress akhirnya terjadi.

Jika asumsi dilanggar ($\geq \frac{1}{3}$ kekuatan Byzantine), keamanan tidak lagi dijamin dan chain mungkin fork. Pemulihan dalam kasus ini memerlukan mekanisme koordinasi sosial—mengidentifikasi chain kanonis melalui sinyal kepercayaan eksternal—seperti dalam sistem BFT mana pun.

4.2 Lapisan Eksekusi

Dua jalur eksekusi beroperasi pada state kanonis yang sama.

Ethereum Virtual Machine. Sentrix menjalankan EVM melalui implementasi Rust berperforma tinggi [4]. Kontrak Ethereum standar (ERC-20, ERC-721, pool ala Uniswap, protokol lending) dapat di-deploy tanpa modifikasi. Tooling standar—Foundry, Hardhat, MetaMask, ethers, dan viem—bekerja langsung. Akuntansi gas mengikuti model EIP-1559: base fee per blok yang menyesuaikan dengan permintaan, plus tip yang ditetapkan pengirim untuk membayar proposer untuk inklusi.

Operasi protokol native. Penerbitan token (SRC-20), staking (delegate, undelegate, claim rewards), dan koordinasi validator bukan smart contract melainkan transaksi yang ditafsirkan langsung oleh protokol. Mereka memakan gas lebih sedikit daripada operasi setara berbasis kontrak karena melewati EVM sepenuhnya. Mereka tidak dapat dieksploitasi melalui kode yang belum diaudit karena perilakunya ditetapkan pada level protokol.

Kedua jalur saling beroperasi. Pengguna yang memegang token native SRC-20 dapat berinteraksi dengan kontrak EVM yang membaca saldo tersebut melalui precompile query saldo kanonis. Kontrak EVM dapat

mengotorisasi operasi native melalui gateway system-call. State kanonis chain adalah penggabungan tunggal dari state EVM dan ledger native.

4.3 Lapisan State

Sentrix memelihara binary sparse Merkle tree sebagai representasi state kanonis [9]. State root dicap ke setiap blok setelah ketinggian aktivasi yang ditentukan, memberikan jaminan kriptografis bahwa setiap node yang menerapkan blok yang sama mencapai state yang sama. State root yang berbeda menghasilkan hash blok yang berbeda, dan BFT memastikan chain konvergen pada satu sejarah kanonis.

State dipersistensikan di MDBX [10], embedded key-value store yang dioptimalkan untuk pembacaan acak throughput tinggi. State penuh bersifat lokal di setiap full node; light client dapat memverifikasi terhadap trie root dengan bukti Merkle logaritmik.

4.3.1 Fungsi Transisi State

Sebuah blok Sentrix menerapkan urutan transaksi ke state sebelumnya untuk menghasilkan state baru. Secara formal, fungsi transisi state $APPLY(S, B)$ mengambil state S dan blok $B = (\text{header}, \text{txs})$ dan menghasilkan state baru S' atau $INVALID$:

$APPLY(S, B)$:

1. Verifikasi header B (parent hash, timestamp, signature proposer).
2. Untuk setiap transaksi TX dalam $B.txs$, secara berurutan:
 - a. Verifikasi signature TX terhadap $TX.from$.
 - b. Verifikasi $TX.nonce == S[TX.from].nonce$.
 - c. Hitung $fee = TX.gas_used \times TX.gas_price$ (jalur EVM) atau MIN_TX_FEE (jalur native).
 - d. Jika $S[TX.from].balance < fee + TX.value$: skip (insufficient).
 - e. Kurangi fee dari $TX.from$.
 - f. Bakar 50% fee ke $BURN_ADDRESS$; bayar 50% ke proposer blok.
 - g. Apply TX :
 - Jalur EVM: invoke $revm$ dengan TX sebagai call ke $TX.to$.
 - Jalur native: dispatch op_type ke handler native.
 - h. Jika apply berhasil: update accounts, increment nonce, emit events.
Jika apply gagal: revert account state, charge fee tetap.
3. Hitung state root baru dari trie yang diupdate.
4. Verifikasi state root baru cocok dengan $B.header.state_root$.
5. Return S' jika semua check lolos; jika tidak, $INVALID$.

Fungsi ini deterministik: setiap node yang mengeksekusi S dan B yang sama menghasilkan S' yang bit-identik. Determinisme inilah yang memungkinkan node-node independen memverifikasi klaim satu sama lain tentang state chain.

4.3.2 Light Client

Light client tidak menyimpan full state. Mereka melacak header blok dan memverifikasi fakta tertentu dengan meminta bukti Merkle dari full node. Diberikan header blok yang berisi state root kanonis, light client dapat memverifikasi setiap saldo akun, slot storage kontrak, atau inklusi transaksi menggunakan bukti berukuran logaritmik terhadap root. Ini membuat penggunaan dompet mobile, dApp browser, dan integrasi sumber daya terbatas praktis tanpa mengkompromikan keamanan: light client mempercayai bukti kriptografis terhadap state root yang telah secara independen diverifikasinya, bukan node yang merespons itu sendiri.

4.4 Model Jaringan

Node Sentrix berkomunikasi melalui mesh libp2p [11]. Protokol menggunakan tiga kelas pesan:

- **Block gossip:** Blok yang baru difinalisasi merambat melalui gossipsub dengan topik `sentrix/blocks/1`. Penerima memverifikasi blok secara lokal dan menerapkannya ke state kanonis.
- **Transaction gossip:** Transaksi yang dikirim pengguna merambat melalui topik `sentrix/txs/1` sampai validator memasukkannya ke blok yang diusulkan.

- **Pesan BFT:** Proposal, prevote, dan precommit adalah request-response langsung antar validator pada topik sentrix/bft/1. Rebroadcast memperkuat pesan yang mungkin telah dijatuhkan.

Penemuan peer menggunakan Kademia DHT [12] dengan alamat seed peer untuk bootstrap. Setiap node juga mempublikasikan record “validator advertisement” yang berisi multiaddr libp2p saat ini, memungkinkan validator lain mempertahankan koneksi langsung tanpa koordinasi eksternal.

Sebuah node yang bergabung dengan jaringan bootstrap sebagai berikut:

1. Connect ke satu atau lebih seed peer (dikonfigurasi saat startup).
2. Run Kademia DHT walk untuk mengisi tabel routing peer.
3. Request chain head dari peer; identifikasi chain stake-weighted terpanjang.
4. Sync block-by-block dari peer sampai head saat ini tercapai.
5. Subscribe ke gossip topics; mulai berpartisipasi dalam BFT jika validator.

Mode kegagalan jaringan di bawah partisi well-defined: partisi minoritas berhenti (tidak dapat mencapai supermayoritas); partisi mayoritas berlanjut dengan active set yang tereduksi. Setelah partisi sembuh, minoritas mendeteksi chain kanonis yang lebih panjang dan rejoin.

4.5 Format Transaksi

Transaksi Sentrix memiliki struktur berikut:

```
Transaction {
  chain_id: uint16,    // 7119 mainnet, 7120 testnet
  nonce:   uint64,    // sequence akun pengirim
  op_type: uint8,      // 0=transfer native, 1=SRC-20, 2=staking, 3=EVM, dll.
  from:    address20, // hex 20-byte berprefix 0x
  to:      address20,
  value:   uint64,    // jumlah dalam sentri (1 SRX = 108 sentri) untuk native
                  // atau wei (1 SRX = 1018 wei) untuk EVM
  gas_limit: uint64,  // hanya jalur EVM; ops native menggunakan
MIN_TX_FEE
  gas_price: uint64,  // wei per gas; hanya jalur EVM
  data:      bytes,   // EVM call data, atau native op payload
  signature: secp256k1, // 65 bytes (r, s, v)
}
```

Diskriminator op_type mengidentifikasi apakah transaksi menargetkan jalur native atau jalur EVM. Signature diverifikasi terhadap payload signing EIP-155 kanonis, memberikan perlindungan replay lintas chain. Ukuran transaksi maksimum dikonfigurasi pada level protokol; transaksi yang ukurannya berlebihan ditolak oleh mempool.

5. Operasi Native

Pilihan desain sentral di Sentrix adalah promosi primitif ekonomi umum dari smart contract menjadi operasi protokol.

5.1 Pertanyaan Native vs Kontrak

Pola standar di chain EVM adalah mengimplementasikan semuanya—termasuk operasi yang paling umum seperti transfer token dan staking—sebagai smart contract. Abstraksinya seragam: semua operasi menelan gas, semua tunduk pada audit keamanan kontrak, semua dapat diprogram. Ini secara konseptual bersih namun mengeluarkan tiga biaya.

Pertama, **overhead eksekusi**. Transfer token sederhana di ruang EVM memerlukan pemuatan bytecode kontrak, dispatch ke selektor fungsi, melakukan baca-tulis storage melalui namespace kontrak, dan emit event. Biaya minimum sekitar 21.000 gas plus eksekusi kontrak. Komputasi sebenarnya—pengurangan saldo pengirim, penambahan saldo penerima—adalah dua operasi storage.

Kedua, **overhead keamanan**. Setiap primitif yang diimplementasikan kontrak harus diaudit secara independen. Kontrak token telah menghasilkan kerugian miliaran dolar melalui bug dalam kode yang sudah lama di-deploy. Operasi native adalah bagian dari konsensus chain, diaudit sekali, deterministik selamanya.

Ketiga, **friksi upgradabilitas**. Primitif yang diimplementasikan kontrak sulit di-upgrade tanpa migrasi terkoordinasi. Primitif native berkembang melalui hard fork, di mana seluruh jaringan upgrade secara atomik.

5.2 Set Operasi Native

Operasi native Sentrix mencakup:

- **Operasi Token SRC-20.** Menerbitkan token fungible, mentransfernya, menyetujui allowance. Diimplementasikan sebagai varian transaksi yang langsung diterapkan ke ledger native. State token tinggal di state trie kanonis bersama saldo akun.
- **Operasi Staking.** Delegate stake ke validator, undelegate (dengan periode unbonding), claim reward yang terakumulasi, register sebagai kandidat validator. Diterapkan langsung ke stake registry.
- **Transfer Native.** Kasus paling sederhana: memindahkan SRX antar akun. Memakan biaya minimum protokol ($\text{MIN_TX_FEE} = 10.000 \text{ sentri} = 0,0001 \text{ SRX}$). 50% dibakar, 50% ke proposer.
- **Koordinasi Validator.** Aktivasi, deaktivasi, dan rotasi validator melalui seleksi berbobot stake. Tidak ada kontrak governance; protokol menerapkan rotasi setiap epoch.

Operasi-operasi ini tetap sepenuhnya dapat diprogram: kontrak EVM dapat memanggil mereka melalui gateway sistem, dan dompet mana pun yang mendukung Sentrix dapat mengeksekusinya melalui satu transaksi yang ditandatangani.

5.3 Siklus Hidup Transaksi

Untuk membuat arsitektur konkret, pertimbangkan transfer SRC-20 100 token dari Alice ke Bob.

Langkah 1 — Susun transaksi

Dompet Alice membangun:

```
op_type  = 1 (SRC-20)
from     = 0xAlice...
to       = 0xTOKEN_CONTRACT
value    = 0
data     = SRC20_TRANSFER || 0xBOB... || 100
nonce    = nonce Alice saat ini
sig      = sign(payload, alice_sk)
fee      = MIN_TX_FEE = 10.000 sentri
```

Langkah 2 — Submit

Dompet mengirim ke endpoint RPC mana pun.

RPC memvalidasi format dasar, menempatkan di mempool.

Mempool gossip tx ke peer via libp2p.

Langkah 3 — Inklusi blok

Validator V (proposer untuk ronde berikutnya) mengurus mempool.

V mengkonstruksi blok N+1 yang berisi tx Alice + lainnya.

V mengusulkan blok N+1 ke active set.

Langkah 4 — Finalisasi BFT

Validator aktif menerima proposal, memverifikasinya.

Setiap validator prevote jika proposal valid.

Setelah $\geq \frac{2}{3}$ prevote teramati, validator precommit.

Setelah $\geq \frac{2}{3}$ precommit teramati, blok N+1 difinalisasi.

Langkah 5 — Apply state

Setiap node menerapkan blok N+1:

- a. Kurangi 10.000 sentri dari akun Alice.
- b. Bakar 5.000 sentri ke BURN_ADDRESS (50% dari fee).

- c. Kreditkan 5.000 sentri ke pending rewards validator V (50% dari fee).
- d. Kurangi saldo token SRC-20 Alice sebesar 100.
- e. Tambahkan saldo token SRC-20 Bob sebesar 100.
- f. Tambahkan nonce Alice.
- g. Emit Transfer event.
- h. Update state trie root; root baru di-commit di block hash.

Langkah 6 — Konfirmasi

Dompot Alice polling RPC; konfirmasi tx di blok yang difinalisasi.
 Dompot Bob (subscribe via WebSocket) menerima notifikasi.
 Total waktu dari Langkah 2 ke Langkah 6: ~1-2 detik.

Siklus ini identik secara prinsip untuk transfer native, panggilan kontrak EVM, dan operasi staking—perbedaannya adalah jalur apply mana yang diambil di langkah 5.

6. Tokenomik

Aset native Sentrix adalah SRX. Kebijakan moneterinya tetap.

6.1 Pasokan

Pasokan maksimum adalah 315 juta SRX. Tidak ada mekanisme governance untuk mengubahnya. Setelah maksimum tercapai melalui block reward, tidak ada SRX baru yang dibuat.

6.2 Block Reward dan Halving

Block reward dasar adalah satu SRX. Reward terbagi separuh setiap kira-kira 126 juta blok (~empat tahun pada waktu blok satu detik), dimodelkan setelah cadensi halving Bitcoin.

Jadwal halving (perkiraan, cadensi empat tahun):

Era	Tahun	Block reward	Total dikeluarkan
1	0-4	1,000 SRX	126 juta SRX
2	4-8	0,500 SRX	189 juta SRX
3	8-12	0,250 SRX	220,5 juta SRX
4	12-16	0,125 SRX	236,25 juta SRX
5	16-20	0,0625 SRX	244,13 juta SRX
...	...	...	(asimtotik)

Dikombinasikan dengan premine 63 juta (Bagian 6.4), pasokan maksimum mendekati 315 juta SRX dalam horizon ~24 tahun, lalu plateau.

6.3 Mekanisme Fee

Setiap transaksi membayar fee. Model fee berbeda berdasarkan jalur eksekusi:

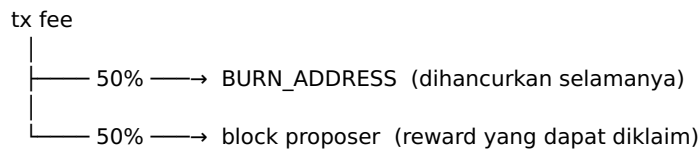
Jalur native. MIN_TX_FEE flat 10.000 sentri (0,0001 SRX) per transaksi, terlepas dari operasi. Operasi native memiliki biaya tetap pada level protokol.

Jalur EVM. Mekanika EIP-1559 [13] standar: baseFeePerGas per blok yang menyesuaikan dengan kongesti, plus tip yang ditetapkan pengirim. $\text{gasUsed} \times (\text{baseFee} + \text{tip})$ dikenakan.

Dari setiap fee yang dibayar, di kedua jalur:

- **50% dihancurkan selamanya** dengan dikirim ke alamat burn yang dapat diverifikasi, dari mana tidak ada transaksi yang dapat diproduksi. Total yang dibakar dapat diamati publik di chain.
- **50% dibayarkan kepada proposer blok** di mana transaksi disertakan. Ini membentuk komponen variabel reward validator di atas subsidi blok tetap.

Gambar 3 — Aliran pembagian fee



6.4 Premine dan Distribusi Awal

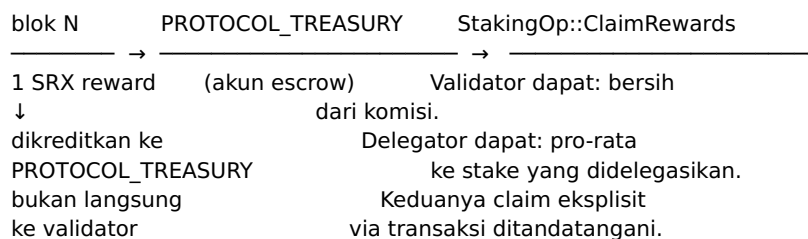
Premine sebesar 63 juta SRX—20% dari cap pasokan—dialokasikan saat genesis di empat peran:

Peran	Jumlah	Tujuan
Founder	21 juta SRX	Treasury, pengembangan awal, kelangsungan operasional
Early Validator	10,5 juta SRX	Bootstrap validator awal dan seeding reward
Ecosystem Fund	21 juta SRX	Hibah, hadiah hackathon, integrasi kemitraan, likuiditas pasar
Reserve	10,5 juta SRX	Cadangan strategis di bawah kontrol multi-signature

Sisa 80% pasokan (252 juta SRX) diterbitkan melalui block reward selama kira-kira 24 tahun.

6.5 Routing Reward

Gambar 4 — Routing reward



Block reward tidak membayar validator secara langsung. Mereka dikreditkan ke escrow treasury protokol, dari mana validator dan delegator mereka mengklaim earning yang terakumulasi melalui operasi staking.

7. Ekonomi Validator

7.1 Stake yang Dipertaruhkan dan Slashing

Setiap validator harus mengikat self-stake minimum ke protokol. Self-stake dipertaruhkan: kemisbehaviour yang dapat dibuktikan menghasilkan slashing. Matriks slashing:

Pemicu	Bukti	Stake di-slash	Durasi jail
Double-sign	Dua signature pada blok yang konflik di height yang sama	20% (parametrik)	Permanen (tombstone)
Downtime	Melewatkan > threshold blok dalam	0,1% (parametrik)	Configurable jail blocks

Penalti double-sign berat karena bukti tidak ambigu dan tindakan bersifat malicious. Penalti downtime ringan karena penyebab sah (reboot kernel, blip jaringan singkat) umum; ia menskala dengan persistensi alih-alih menghukum ketidakhadiran transien. Kedua penalti mengurangi stake aktif validator dan menghapusnya dari active set; re-entry membutuhkan unjail eksplisit.

SRX yang di-slash dihancurkan (ditambahkan ke BURN_ADDRESS), bukan didistribusi ulang. Ini menjaga invarian pasokan dan menghindari menciptakan insentif buruk di antara validator yang tidak di-slash untuk mendorong slashing kompetitor.

7.2 Block Reward dan Bagi Hasil Fee

Validator memperoleh block reward (subsidi tetap) plus bagi hasil fee (50% dari setiap fee transaksi di blok yang mereka usulkan).

Delegator mereka memperoleh secara proporsional dengan stake yang didelegasikan, dikurangi komisi yang ditetapkan oleh validator.

Pendapatan harapan validator per epoch:

$$\text{expected_revenue} = (\text{block_subsidy} \times \text{blocks_proposed}) + (\text{fee_share} \times \text{tx_fees_in_those_blocks})$$

di mana:

$$\begin{aligned}\text{blocks_proposed} &\approx \text{epoch_length} \times (\text{validator_stake} / \text{total_active_stake}) \\ \text{fee_share} &= 0,5\end{aligned}$$

7.3 Penalti Liveness

Validator yang melewatkan blok mengakumulasi downtime terhadap window yang bergerak (LIVENESS_WINDOW). Downtime berkelanjutan menghasilkan jailing, yang menghapus validator dari active set dan slashing sebagian kecil stake.

7.4 Genesis dan Bootstrap

State genesis Sentries dikonstruksi dari file konfigurasi (genesis.toml) yang mendeklarasikan:

- Chain ID (7119 mainnet, 7120 testnet).
- Saldo premine awal (alokasi 63 juta SRX di empat peran).
- Validator set awal dengan public key dan komitmen self-stake.
- Parameter protokol efektif dari blok 1 (block time, konstanta fee, fork heights).

Node mana pun dapat memverifikasi state genesis dengan menerapkan ulang konfigurasi genesis; state root yang dihasilkan harus cocok dengan hash genesis yang hardcoded.

8. Model Keamanan

8.1 Jaminan Kriptografis

Setiap transaksi ditandatangani dengan kunci privat yang alamat publiknya mengotorisasi operasi. Header blok berisi state root dan signature proposer. State root commit ke state kanonis via binary sparse Merkle tree.

8.2 Jaminan Kripto-Ekonomi

Validator berpartisipasi karena menguntungkan untuk melakukannya secara jujur. Slashing menjadikan validasi tidak jujur memiliki nilai harapan negatif pada ukuran stake mana pun. Tarif slashing dikalibrasi untuk membuat serangan safety-violating yang berhasil membutuhkan komitmen stake terkoordinasi setidaknya sepertiga

dari total—komitmen stake yang biaya marjinalnya ($X \cdot P/3$ di mana X adalah total bonded SRX dan P adalah harga pasar SRX) tumbuh dengan kesuksesan chain.

8.3 Jaminan Sosial

Perilaku chain dapat diamati. State, blok, transaksi, validator set, peristiwa slashing—semuanya publik. Trust digantikan oleh verifikasi.

8.4 Keamanan Kuantitatif

Biaya untuk mengompromikan keamanan chain dibatasi ke bawah oleh biaya untuk mengakuisisi dan slashing sepertiga dari total bonded SRX. Jika protokol mengikat X SRX pada harga pasar P , maka biaya minimum serangan A memenuhi:

$$A \geq (X \times P) / 3$$

Ini batas bawah, bukan batas atas: serangan praktis menghadapi biaya tambahan (mengkoordinasikan akuisisi stake tanpa gangguan pasar, mengeksekusi serangan dalam window unbonding, menerima kerusakan reputasi permanen). Batas naik secara monoton dengan baik X (pasokan bonded) dan P (harga pasar), yang berarti kesuksesan chain meningkatkan biaya serangan.

Sebagai contoh kerja: jika 100 juta SRX di-bond pada harga pasar \$0,10/SRX, biaya minimum serangan adalah sekitar \$3,3 juta. Pada 200 juta bonded dan \$1,00/SRX, biayanya naik ke ~\$67 juta. Pada 300 juta bonded dan \$5,00/SRX, biayanya ~\$500 juta. Angka-angka ini ilustratif; nilai aktual tergantung dinamika pasar.

8.5 Long-Range Attacks dan Weak Subjectivity

Chain proof-of-stake murni secara teoretis rentan terhadap long-range attacks: penyerang yang mengakuisisi kunci validator lama (setelah pemiliknya unbond dan menjual) dapat fork chain dari titik awal yang sewenang-wenang. Sentrix bertahan terhadap ini melalui tiga mekanisme:

- **Periode unbonding.** Stake dapat di-slash untuk durasi tetap setelah penarikan. Penyerang yang mengakuisisi kunci harus memalsukan blok sebelum periode unbonding berlalu.
- **Weak subjectivity checkpoints.** Node baru yang bergabung dengan jaringan diharapkan mulai dari hash blok terbaru yang mereka percayai melalui beberapa channel out-of-band (peer, validator yang dipercaya, website proyek).
- **Sync active-set periodik.** Light client melakukan re-sync validator set yang dipercaya pada interval lebih pendek dari periode unbonding.

Mekanisme-mekanisme ini standar untuk chain proof-of-stake BFT [14].

8.6 Mode Kegagalan

Chain memiliki empat mode kegagalan well-defined:

- **Lebih dari $\frac{1}{3}$ stake Byzantine.** Keamanan tidak lagi dijamin. Chain mungkin fork. Pemulihan memerlukan koordinasi sosial.
- **Lebih dari $\frac{2}{3}$ stake Byzantine.** State invalid dapat diproduksi dan ditandatangani. Deteksi memerlukan full node jujur memverifikasi state secara independen.
- **Partisi jaringan.** Partisi minoritas berhenti karena tidak dapat mencapai supermayoritas. Partisi mayoritas berlanjut dengan active set tereduksi. Ketika partisi sembuh, minoritas mendeteksi chain kanonis yang lebih panjang dan rejoin.
- **Sensor terkoordinasi terhadap transaksi spesifik.** Tahan terhadap sensor satu kali karena rotasi proposer menjamin bahwa validator aktif yang tidak menyensor pada akhirnya akan

mengusulkan blok. Sensor berkelanjutan membutuhkan kontrol active validator set, yang menurut seleksi berbobot stake membutuhkan kontrol stake mayoritas.

9. Tesis Ekonomi Riil

Posisi Sentrix adalah penyelesaian dunia riil.

Aset dunia riil. Hak atas sesuatu yang tangible dan kontraktual—real estate, faktur, ekuitas di perusahaan swasta, hak atas aliran pendapatan, bill of lading—berharga bagi pemiliknya. Mereka juga tidak likuid. Blockchain yang dapat merepresentasikan ini sebagai token on-chain, dengan provenance yang dapat dibuktikan dan penyelesaian atomik, menghilangkan friksi proporsional dengan nilai aset.

Penyelesaian mata uang lokal. Sebagian besar aktivitas ekonomi didenominasi dalam mata uang lokal—rupiah, peso, dong, baht, ringgit, dirham. Blockchain yang mendukung penerbitan stablecoin terhadap mata uang ini, dengan operasi native untuk transfer murah cepat, menjadi rel penyelesaian untuk perdagangan retail yang saat ini tidak memilikinya.

Perdagangan lintas-negara dan remitansi. Pembayaran lintas-negara secara tradisional diselesaikan melalui correspondent banking dengan keterlambatan beberapa hari dan fee yang tidak proporsional. Blockchain yang operasi native-nya memakan biaya pecahan sen dan finalisasi dalam satu detik secara fundamental cocok untuk aliran lintas-negara berukuran kecil. Indonesia adalah negara penerima remitansi pada skala puluhan miliar dolar per tahun.

Microfinance dan penyelesaian koperasi. Tabungan kelompok (arisan), perbankan koperasi (koperasi simpan-pinjam), dan pinjaman komunitas tertanam dalam dalam budaya ekonomi Indonesia. Mereka beroperasi pada kepercayaan, catatan kertas, dan koordinasi informal. Blockchain yang dapat merepresentasikan aliran ini—on-chain, dapat diverifikasi, biaya rendah—adalah upgrade alih-alih pengganti.

10. Perbandingan dengan Karya Sebelumnya

	Block time	Finalitas	Konsensus	VM	Kebijaksanaan pasokan
Bitcoin [1]	~10 menit	Probabilistik	Proof of Work	Tidak ada	Capped halving
Ethereum [2]	~12 detik	Probabilistik → final	Proof of Stake	EVM	Inflasi (variabel)
Solana [5]	~400 ms	Probabilistik	PoH + TowerBFT	SVM (BPF)	Inflasi (menurun)
Cosmos Hub [3,6]	~6 detik	Single-block	Tendermint BFT	Cosmos SDK (Go modules)	Inflasi
Polygon	~2 detik	Probabilistik + checkpointed	Varian PoS BFT	EVM	Inflasi (capped)
Sentrix	~1 detik	Single-block	DPoS + BFT (ala Tendermint)	EVM (revm) + Native rail	Capped halving 50% burn

Sentrix tidak baru dalam dimensi tunggal mana pun. Kontribusinya adalah kombinasi: eksekusi hibrida native-EVM, BFT berbobot stake, pasokan capped deflasi, finalitas sub-detik, berorientasi pada penyelesaian dunia riil dan berakar di pasar Indonesia.

11. Arah ke Depan

Kami menjelaskan trajektori Sentrrix dalam bahasa tahap alih-alih tanggal, karena tanggal menciptakan kekhususan palsu yang kondisi riil tidak pernah hormati.

Tahap 1: Mainnet beroperasi. Validator set kecil yang terdistribusi secara geografis. Eksekusi native dan EVM bekerja dengan benar. Block explorer, faucet, dev tooling. Codebase source-available. (*Status saat ini.*)

Tahap 2: Likuiditas dan penemuan. Pasar on-chain pertama untuk SRX. Kontrak pertama yang di-deploy oleh developer eksternal. Indexing di registry chain standar. Pengakuan oleh dompet dan bridge standar. Komunitas awal.

Tahap 3: Integrasi ekonomi riil. Kasus penggunaan produksi pertama yang melibatkan aset dunia riil, penyelesaian mata uang lokal, atau aliran lintas-negara. Spesifikasi muncul dari penemuan pasar.

Tahap 4: Desentralisasi validator. Pertumbuhan validator set dari bootstrap kecil ke jumlah operator independen yang bermakna di berbagai yurisdiksi.

Tahap 5: Governance on-chain. Migrasi keputusan protokol yang berarti ke mekanisme governance berbobot stake.

Tahap 6: Aktivasi lisensi terbuka. Codebase bertransisi dari source-available ke fully open source di bawah ketentuan standar.

Tahap-tahap ini tidak berjalan pada kalender. Mereka berjalan pada kepuasan prasyarat.

12. Kesimpulan

Sentrrix adalah respons yang disengaja terhadap absennya struktur. Blockchain dominan saat ini melayani perdagangan dan spekulasi dengan baik, dan mereka melayani penyelesaian ekonomi riil dengan buruk. Kami tidak percaya ini tak terhindarkan. Kami percaya ini adalah konsekuensi dari pilihan desain yang dapat dibuat berbeda.

Pilihan desain Sentrrix adalah: operasi native untuk primitif umum, kompatibilitas EVM untuk kasus umum, konsensus Byzantine Fault Tolerant berbobot stake, finalitas sub-detik, pasokan capped halving deflasioner, dan orientasi pintu masuk pasar terhadap ekonomi Indonesia.

Kami berharap untuk beroperasi dalam jangka waktu yang lama. Sentrrix terbuka untuk digunakan, terbuka untuk diperluas, dan terbuka untuk diperiksa. Nilai riil, aset riil, aktivitas ekonomi riil—on chain, cepat, final, dan tahan lama.

Lampiran A — Parameter Protokol

Parameter	Nilai	Catatan
BLOCK_TIME	~1 d	Target; aktual bervariasi dengan durasi ronde
MAX_TX_PER_BLOCK	5.000	Configurable pada level protokol
MAX_SUPPLY	315.000.000 SRX	Hard cap, tidak ada governance override
INITIAL_BLOCK_REWARD	1 SRX	Halving setiap HALVING_PERIOD
HALVING_PERIOD_BLOCKS	~126.000.000	~4 tahun pada blok 1 d 0,0001 SRX (jalur

MIN_TX_FEE	10.000 senti	native)
BURN_RATIO	50%	Dari setiap fee transaksi
MIN_VALIDATOR_SELF_STAKE	parametrik	Dikonfigurasi di genesis
UNBONDING_PERIOD	parametrik	Window stake-slashable setelah penarikan
LIVENESS_WINDOW	14.400 blok	~4 jam pada blok 1 d
MIN_SIGNED_PER_WINDOW	4.320 blok	30% dari LIVENESS_WINDOW
JAIL_DURATION_BLOCKS	600	~10 menit pada blok 1 d
DOWNTIME_SLASH_BP	10 (0,1%)	Dari self-stake saat jail
EPOCH_LENGTH	parametrik	Cadensi rotasi active-set
STATE_ROOT_FORK_HEIGHT	aktif	State root di-commit di hash blok

Referensi

- [1] Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*.
- [2] Buterin, V. (2014). *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*.
- [3] Buchman, E., Kwon, J. (2016). *Tendermint: Byzantine Fault Tolerance in the Age of Blockchains*.
- [4] Bluealloy (2022). *revm: Rust Ethereum Virtual Machine*.
- [5] Yakovenko, A. (2017). *Solana: A new architecture for a high performance blockchain*.
- [6] Kwon, J., Buchman, E. (2019). *Cosmos: A Network of Distributed Ledgers*.
- [7] Fischer, M., Lynch, N., Paterson, M. (1985). *Impossibility of Distributed Consensus with One Faulty Process*.
- [8] Castro, M., Liskov, B. (1999). *Practical Byzantine Fault Tolerance*.
- [9] Merkle, R. (1980). *Protocols for Public Key Cryptosystems*.
- [10] MDBX. *Memory-mapped key-value store*. <https://github.com/erthink/libmdbx>
- [11] libp2p. *Modular peer-to-peer networking stack*. <https://libp2p.io>
- [12] Maymounkov, P., Mazières, D. (2002). *Kademlia: A Peer-to-peer Information System Based on the XOR Metric*.
- [13] Buterin, V. dkk. (2019). *EIP-1559: Fee market change for ETH 1.0 chain*.
- [14] Buterin, V., Griffith, V. (2017). *Casper the Friendly Finality Gadget*.
- [15] Dwork, C., Lynch, N., Stockmeyer, L. (1988). *Consensus in the Presence of Partial Synchrony*.

Penghargaan

Sentrix adalah produk kerja seorang penulis tunggal selama periode yang sengaja dipadatkan. Penulis mengakui para insinyur yang membangun fondasi tempat Sentrix dikonstruksikan—core developer

Ethereum, tim Cosmos dan Tendermint, komunitas Bitcoin, ekosistem Rust, para maintainer open-source dari pustaka kriptografis dan jaringan yang membuat proyek dengan cakupan ini feasible bagi seorang individu untuk mengambilnya.

Keputusan untuk membangun Sentrax dalam Rust, di atas EVM, dengan konsensus BFT ala Tendermint, tidak memerlukan penemuan komponen-komponen ini. Ia hanya memerlukan komposisinya. Inilah cara infrastruktur yang tahan lama dibangun: bukan dari ide-ide baru, melainkan dari penyusunan ide-ide yang sudah ada secara cermat.